2026-10-20 17:07:32 **You**

best way to ask codex to build from design docs something similar to notebookllm diagrams or infographs but whilke being creative interactive web content like claude code artifacts

displaying a game development design as visual as posible, workflows, graphs, charts, whatever

ChatGPT gpt-6-thinking

I'll put together a copy-paste Codex prompt for turning game design docs into an interactive visual experience, plus a recommended stack and a way to keep Codex from producing a generic dashboard.

The best approach is to ask Codex to create an **Interactive Game Design Atlas**, not merely a diagram generator.

Think of it as a combination of:

- **NotebookLM:** Automatically understands design documents and turns complex ideas into clear visual explanations.
- **Claude Artifacts:** Creates beautiful, interactive, explorable web experiences.
- **Miro / Figma:** Lets you navigate complex systems spatially.
- **Game design tools:** Visualizes gameplay loops, progression, economies, dependencies, state machines, and even small playable simulations.

The important difference is that Codex should **interpret each game mechanic and invent the most useful visual representation for it**, rather than forcing everything into flowcharts and dashboard cards.

For example, a combat system might become an interactive combat simulator, while a progression system becomes an unlockable skill tree. A resource economy might become an animated production-and-consumption network.

This fits particularly well with OpenAI's frontend design guidance, which explicitly encourages expressive, animated, playful interfaces for games rather than generic application dashboards.

OpenAI API

The stack I'd recommend

Technology	Purpose

React + TypeScript + Vite	Fast, standalone interactive web application
Tailwind CSS + shadcn/ui	Polished interface and controls
React Flow	Interactive skill trees, workflows, dependency networks
Recharts	Progression curves, balancing charts, resource economies
D3.js / custom SVG	Unique visualizations that aren't standard charts
Motion	Smooth transitions and meaningful animations
tldraw (optional)	Infinite canvas, freeform exploration and annotations

React Flow already provides node dragging, zooming, panning and selection; tldraw is better when the whole experience should behave like a creative whiteboard. React Flow +2

I would start with React Flow as the main spatial visualization engine, and add the others only where they genuinely improve understanding.

Copy-paste this into Codex

Use it from the root of your game project, where Codex can access your design documents.

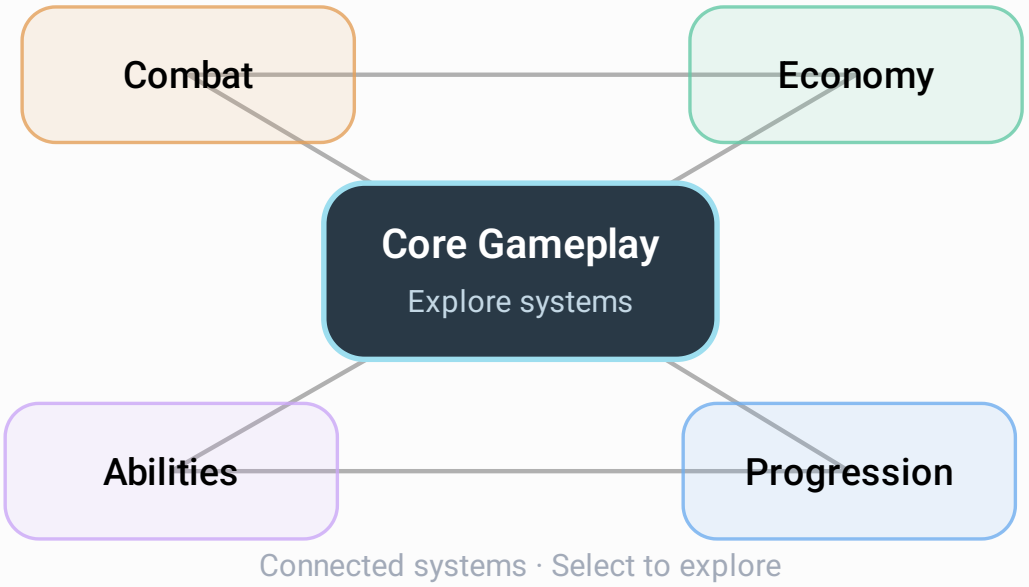
MASTER PROMPT — GAME DESIGN ATLAS

Copy prompt

The full prompt includes document analysis, visualization design, interaction requirements, tech stack, implementation steps, and acceptance criteria. Use the copy button to paste it into Codex.

The result you should be aiming for

Game Design Atlas Concept



Progression Explorer

Interactive unlock trees, prerequisite tracing, character development.

Systems Lab

Change hypothetical parameters to examine mechanics and outcomes.

Balance Analytics

Compare scaling, resource dynamics, costs, and progression.

Player Journey

Explore player decisions, milestones, loops, and branches.

Illustrative interface structure, not an application generated from your design documents.

An important second prompt

After Codex builds the first version, I would run a separate visual-quality pass. This is where you can push it toward the kind of surprising, polished results you're describing.

Creative refinement prompt Copy

Now act as a demanding creative director and information visualization specialist. Review the actual application, not just its code. Identify every area that feels like a generic AI dashboard, documentation website, static flowchart, or ordinary card layout. Replace weak representations with more imaginative, domain-specific, genuinely interactive visual experiences. Prioritize understanding, originality, excellent typography, readable diagrams, responsive layouts, and meaningful motion. Select the three most important game systems and make their visualizations exceptional. Run and inspect the result, test real interactions, and iterate on visual quality without inventing undocumented game mechanics. Do not just describe improvements; implement them.

One more useful idea: once you have something you like, move the persistent visual direction and document-handling rules into the repository's `AGENTS.md`. Codex supports that file as project-level guidance, so you don't need to re-explain the experience for every subsequent task. OpenAI Develo...

The key principle: Don't ask Codex to *visualize your documentation*. Ask it to **turn your game design into an explorable, interactive model of the game**. That framing gives it substantially more creative freedom while preserving the original design intent.

2026-10-20 17:12:19 **You**

what about using a skill for this? there are already good skills or better to build our own?

or without skills is better? just the whole prompt + the source docs tagged

give me the claude version, or its the same?

ChatGPT gpt-6-thinking

I'll check the current Claude Code and Codex skill options, especially any official frontend-design skills, then give you a Claude-specific setup and prompt. I'll also separate what belongs in a reusable skill from what should stay in the project docs.

I'd use a hybrid approach: existing Anthropic skills + a small custom game-design visualization skill + your source documents.

And there's a particularly interesting development: **Claude Code now supports interactive Artifacts directly**, so you can ask it to create a shareable visual experience without necessarily building and maintaining a full web application. Claude Code D...

For your game design visualization idea, I'd experiment with Claude Code first.

1. The existing skills worth using



[1. Frontend Design](#) Start here

Official Anthropic skill focused on distinctive interfaces, typography, composition, motion and avoiding generic-looking AI dashboards.

This is the one I'd prioritize for visual quality. GitHub



2. Web Artifacts Builder

Another official Anthropic skill. Builds complex React-based interactive experiences and packages them as standalone HTML artifacts.

Useful for experiments with multiple views, state and controls. Less necessary when building an ordinary persistent Vite app. GitHub



3. Interactive Diagram

Community-maintained skill for generating self-contained HTML diagrams with panning, zooming and clickable elements.

Good for technical workflows and system relationships, but narrower than the full experience you described. Review its code before installing. GitHub

ontend Design first, Web Artifacts Builder when needed, and your own game-specific skill once you've discovered what actually works.

2. Prompt vs. skill vs. project instructions

Method	Best use	My verdict
Prompt + design docs	First prototype, one-off visualization	Best place to start
Existing frontend skill + prompt	Improving creative quality	Strongly recommended
Custom `SKILL.md`	Repeatedly visualizing mechanics or updating the atlas	Best long-term
`CLAUDE.md`	Project paths, rules, conventions and constraints	Useful alongside skills

Skills aren't inherently smarter than prompts. Their advantage is that reusable instructions are loaded when relevant, instead of pasting a large master prompt repeatedly. Claude Code supports project-local `.claude/skills/` and personal `~/.claude/skills/`. Claude Code D…

A skill should contain **how to interpret and visualize game systems**. Your design documents should contain **what the game actually is**. Keep those separate.

3. Install the official Claude frontend skill

Inside Claude Code, run:

```
</> Plain text

/plugin install frontend-design@claude-plugins-official
```

If the official marketplace isn't configured, Claude Code provides a command to add it. GitHub +1

You can then explicitly request the skill during frontend work.

4. My Claude Code version of the prompt

I would make one change to the Codex prompt: **prioritize a highly creative, self-contained interactive artifact for the first iteration**, instead of immediately developing a large React application.

That gives you an opportunity to explore possible visual representations before investing in permanent architecture.

Claude Code — Interactive Game Design Atlas Copy

Assumes your documents are in `@docs/`. Replace that path if needed.

Claude Code's artifacts are single-page, self-contained experiences without a backend. They're great for interactive visual explanations, but a local web app is a better home for an atlas that will grow to hundreds of game systems or require complex persistence. Claude Code D…

5. When you're ready: create your own skill

Instead of another enormous prompt, I'd give Claude a **small reusable visualization skill**.

Save this as:

`PROJECT/.claude/skills/game-design-visualizer/SKILL.md`

</> Markdown

```
---
name: game-design-visualizer
description: Turn game design documents into creative, interactive visual explanations,
systems maps, simulations, infographics, and explorable game design atlases. Use for
visualizing game mechanics, progression, economy, combat, and design relationships.
---

# Game Design Visualizer

Transform game design specifications into highly visual, interactive experiences.

## Workflow

1. Discover and read the relevant game design documents.
2. Extract mechanics, entities, relationships, rules, formulas, and dependencies.
3. Classify each concept by how it behaves: sequence, network, hierarchy, state machine,
economy, spatial system, or quantitative model.
4. Choose the most expressive visualization for each concept.
5. Create interactive visualizations grounded in source documents.
6. Verify factual fidelity, functionality, clarity, and visual quality.
```

```
## Creative principles

- Show behavior, causality, dependencies, and consequences.
- Favor meaningful interaction over static diagrams.
- Create bespoke visual metaphors when conventional charts are inadequate.
- Let the game's themes influence the visual language.
- Use animation to explain transitions, not merely decorate.
- Prefer an immersive overview plus focused interactive deep dives.
- Avoid generic dashboards and repetitive card layouts.
- Use the frontend-design skill when available.

## Fidelity

- Treat game documents as authoritative.
- Preserve documented values and formulas.
- Label assumptions, extrapolations, and hypothetical examples.
- Include traceable source references.
- Never silently invent mechanics.

## Delivery

For quick explorations, prefer a self-contained interactive artifact.

For persistent or extensive projects, prefer a standalone React/TypeScript app.

Ensure navigation and interactions work. Inspect the rendered output and improve weak
visualizations before completion.
```

After that, your regular Claude Code prompt can become:

```
</> Plain text

/game-design-visualizer

Explore @docs/ and create an interactive visual
atlas of the game's core mechanics.

Prioritize the most important systems, creative
visual explanations, and real interactivity.

Use the frontend-design skill where available.
```

That's substantially easier to reuse, and it separates the recurring creative methodology from your actual game specifications.

6. What I would actually do

- **First attempt: Claude Code + Frontend Design + full prompt**

  Don't build a custom skill yet. See what Claude produces from your real documents.

- **Second attempt: critique and refine**

  Select the best visualizations. Ask Claude to improve the weak ones and develop a recognizable visual language for your game.

- **Third attempt: turn the successful methodology into a skill**

  Capture the visual patterns, interaction conventions and quality requirements that actually produced good results.

One last point: the most interesting result may not be a giant network graph. It could be a **set of interactive miniature models of your game**, where you can click abilities, simulate progression, adjust resource production, or see systems react to each other.

That's where I think the biggest advantage over NotebookLM-style infographics lies: not just *seeing* the design, but **experimenting with how it works**.

2026-10-20 17:20:36 **You**

list the best projects that like [https://github.com/LizardLiang/interactive-diagram] (https://github.com/LizardLiang/interactive-diagram)

im also building this project:
Represent is the umbrella for tools, POCs, and products that turn source code (and other structured inputs) into artistic visual representations — dependency graphs, creatures, cities, blueprints, animated minimaps, 3D meshes. A project should have an iconic visual identity that reflects its real structure, not just a logo. Developers should look at the picture and understand relationships they'd otherwise have to read thousands of lines to recover.

so i need github project that already do this kind of work, the bests

then list paid private products that also related and we could at least test to learn from them

this project is borning so give also your recommendations

ChatGPT gpt-6-thinking

I'll look for projects that actually turn code structure into visual systems—not just generic graph libraries—then separate the strongest open-source references from paid products worth testing. I'll also rank them by how useful they are for Represent's more ambitious "code as creatures/cities/worlds" direction.

The strongest references are splitting into two camps: projects like CodeCharta and Git City already turn repositories into explorable worlds, while GitNexus and CodeBoarding are stronger at recovering real code relationships. I also found a few procedural-generation projects that could help with Represent's creature and mesh ideas.

Represent — the best existing projects to study

Your description of **Represent** points to something more interesting than another codebase visualizer:

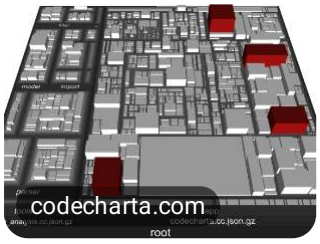
Source code becomes a visual world with its own identity. Its shape, layout, movement, and appearance communicate the structure and behavior of the underlying software.

I researched this from three directions: existing code-to-visualization projects, interactive diagram engines, and commercial products you can test.

My main finding is that there are already strong implementations of individual pieces of Represent, but the opportunity is in **combining accurate structural analysis with distinctive, interactive visual metaphors**.

1. Best open-source projects: code becomes a world

These are the closest matches to your vision, ranked by how valuable I think they are as references for Represent—not simply GitHub stars.



1. CodeCharta Essential

Codebase → 3D city

Files become buildings, folders become districts, and software metrics control their appearance. It supports complexity hotspots, historical comparison and even 3D-printable models.

This is probably your most valuable reference for turning structural data into something spatial, understandable and tangible. CodeCharta

Website and demo



2. Git City

Git repository → living city or farm

Files become buildings—or crops and farmland. You can replay repository history as the environment changes, with a real Git client integrated into the visualization.

Particularly interesting for Represent: it combines metaphor, time, interaction and useful developer workflows instead of stopping at a pretty picture.

GitHub



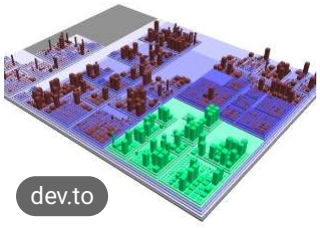
3. Gource

Git history → animated ecosystem

A repository becomes a living tree of branches and files. Contributors move through it, modifying nodes over time.

Arguably one of the best references for representing software evolution as an animation. Study its visual storytelling and how the passage of time becomes understandable without reading a commit log. GitHub

Website

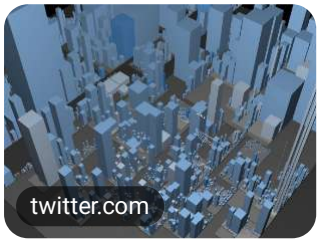


4. JSCity

JavaScript → architectural city

Folders and files form neighborhoods, functions become buildings, and nested functions become structures on top of buildings. Height and footprint encode code metrics.

Especially valuable because it explores how to represent code below the file level. It's an older research project, so treat it as inspiration rather than a modern production foundation. GitHub



5. GoCity

Go source → navigable 3D city

Files, folders and structs become spatial objects. The React/Babylon.js frontend is particularly relevant if you want to investigate procedural 3D representations.

Useful for studying deterministic mappings between code metrics and geometric features. GitHub



6. Git City — developer profiles

GitHub activity → customizable pixel-art buildings

Different from the project above: this transforms developer activity into a global city, with buildings that grow according to contribution metrics and an explorable, game-like environment.

Not a code-architecture analyzer, but extremely relevant for iconic visual identity, gamification and shareable representations. GitHub

Live experience

2. Best GitHub projects for understanding code through graphs

These are less visually artistic than the first group, but extremely valuable for the **structural intelligence layer** of Represent.

Project	What it does	Why I'd study it
GitNexus	Turns code into a knowledge graph of symbols, dependencies, calls, clusters and execution flows	Top priority for analysis. A strong reference for extracting meaningful structures that multiple visual renderers could share. GitHub
CodeBoarding	Static analysis + LLM-assisted architecture extraction, with nested diagrams and source references	Excellent for semantic abstraction and navigating from overview to specific components. GitHub
GitDiagram	Generates interactive architecture diagrams and explainer videos from GitHub repositories	Study its immediate repository-to-visual experience and low-friction sharing. GitHub
CodeFlow	Browser-based interactive architecture maps from repositories	Worth studying for fast, local-first exploration and simple onboarding. GitHub
Emerge	Dependency graphs with modularity clustering, metrics, heatmaps and interactive exploration	Especially interesting for automatically grouping related code into meaningful visual regions. GitHub
Cartograph	AST-derived dependency maps, blast-radius analysis, React Flow interface	Useful implementation reference for trustworthy edges and showing what changing a module might affect. GitHub
Sourcetrail	Interactive symbol-level explorer linking graph navigation to source code	Excellent UX reference for traceability, although the original project was archived in 2021. GitHub

A useful distinction for Represent

GitNexus and CodeBoarding aim to answer *what the software actually does and how its pieces connect*.

CodeCharta and Git City aim to answer *what the software looks like when represented spatially*.

Combining those strengths would give Represent a more powerful foundation than relying on an LLM to invent a scene directly from source files.

3. Projects closest to interactive-diagram: agent skills and visual generators

Since you specifically mentioned interactive-diagram, these deserve their own category.

Oh-my-mermaid

Agent-powered repository scanning that generates hierarchical architecture documentation and navigable diagrams. Works with Claude Code, Codex and Cursor.

Study its skill design, recursive perspectives and document organization. GitHub

---

Eraser Diagrams

Open-source framework for generating polished diagrams, including routing, styling, iconography and export.

Worth exploring as an agent-friendly output engine rather than creating a diagram renderer entirely from scratch. eraserlabs/er…

Interactive Diagram

Generates standalone HTML with multiple diagram types, pan/zoom, detail panels, highlighting and layout checks.

What I'd borrow is its **template-driven architecture**: the agent fills a structured configuration, while a reusable renderer handles layout and interaction. GitHub

For Represent, I would strongly favor that last architectural idea. An AI agent should produce a structured model or visualization specification, **not write thousands of lines of bespoke rendering code for every repository**.

4. Experimental projects for creatures, galaxies and procedural worlds

These aren't necessarily software analyzers. They're references for making Represent's outputs more imaginative.



This Alien Does Not Exist

Deterministically generates unique creatures from numerical seeds. The same seed produces the same creature.

This is close to the concept of a permanent visual fingerprint for a software project. GitHub

---



City Tour

A procedural city with terrain, roads, lots, buildings and navigable camera controls.

Important for learning how to generate a coherent city from rules, rather than simply arranging boxes in 3D. GitHub



GitHub Universe 3D

Turns repository data into planets in an explorable universe, including a spaceship, constellations and cinematic navigation.

an interesting possibility: a repository could have multiple consistent representations—a graph for inspection, a city for exploration, and a creature for instant visual recognition. Better for studying playful interaction and memorable presentations than deep code-level accuracy. GitHub

5. Best paid/commercial products to test

These are actual products, not just GitHub repositories. Some are commercial-only; others offer open-source/community editions alongside paid tiers.

Prices are published starting points as of October 2026 and may vary with billing and licensing.

Product	What to study	Test access / pricing
CodeViz	AI-generated code architecture visualizations, natural-language queries, PR visualization	Free: 5 diagrams/month; Pro: $19/month. Visualize Your ...
Eraser	Visual aesthetics, AI diagram generation from repositories, editing, exporting and collaboration	Free: 3 AI diagrams; Starter: $15/member/month annually. Eraser +1
SciTools Understand	Deep source-level graphs, call relationships, dependency tracing, structural metrics	Full-featured trial; paid from roughly $100/month annually. Free for qualifying public open-source repositories. SciTools +1
CodeScene	Technical debt hotspots, change coupling, code health and evolution	14-day trial; Standard from €18/active author/month billed yearly. CodeScene +1
AppMap	Interactive diagrams generated from actual runtime behavior and execution traces	Free community edition; Team support $15/developer/month. AppMap
IcePanel	Hierarchical software modeling, C4 architecture views, relationships and visual storytelling	Free plan; Growth from $40/editor/month annually. Primarily a modeling tool, not automatic source-code reverse engineering. IcePanel
CAST Imaging	Enterprise-scale code-to-architecture mapping, dependencies, data flows and impact analysis	30-day Express trial; Express listed around €2,000/year. Trial Request +1

My test priority

Eraser Test first

Best place to investigate polished visual output, diagram generation from context and editable visual artifacts.

SciTools Understand

Probably the most valuable commercial reference for asking whether Represent's graphical relationships are structurally accurate, especially at symbol level.

CodeScene

Strong reference for turning historical repository data into meaningful visual signals, rather than just showing current dependencies.

AppMap

Especially interesting if you want animated interactions or flows that represent what an application actually does when it runs.

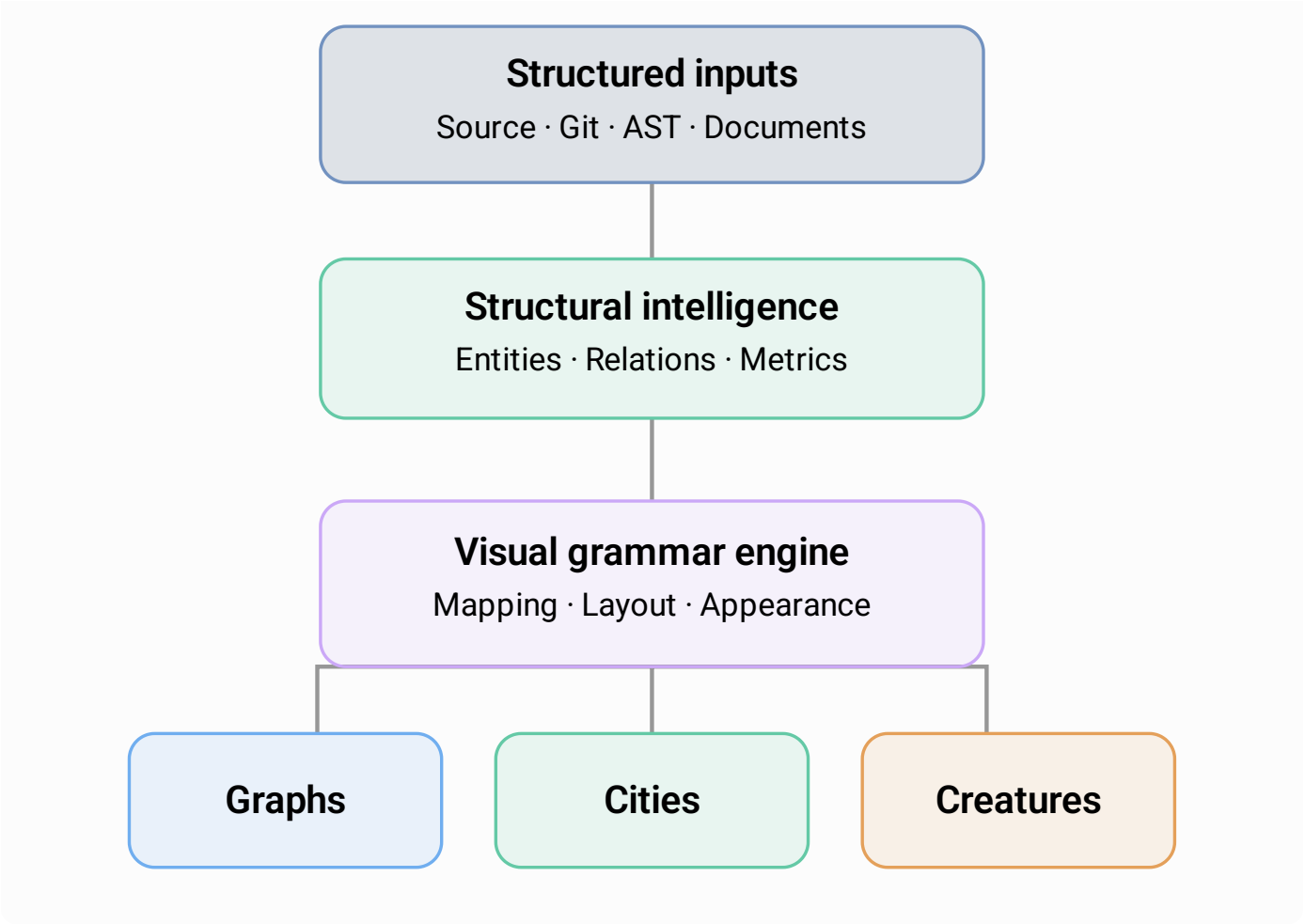
One historical product worth knowing is **CodeSee Maps**. It previously focused on visual codebase maps, but its commercial operations shut down following the GitKraken acquisition. I wouldn't plan a current product test around it. 127 comm… +1

---

6. What I would build for Represent

I wouldn't start by trying to create every visualization type. I'd build **one shared structural engine and three distinctive representations**.

Represent architecture



One source of structural truth, multiple interchangeable ways to explore or recognize it.

The analysis and the visual presentation should be separate. A city renderer, creature generator and dependency graph should all consume the same underlying entity IDs and relationship data.

Three representations worth making exceptional



Represent Graph — understanding

A zoomable, explorable network of real dependencies and logical clusters. Clicking a node reveals callers, dependencies, architectural responsibility and source locations.

Build this first because it establishes trust in the underlying structural model.



Represent City — exploration

A navigable architectural world in which large modules become districts, classes become buildings, and meaningful dependencies become routes or transport links.

Show how the city changes between Git commits. Highlight affected areas when selecting a pull request.



Represent Creature — identity

Every codebase generates a distinctive procedural creature, using a stable mapping from structural metrics to anatomy, proportions, patterns and animation.

This is the most experimental idea, but potentially the most recognizable part of your project.

7. The design decision that matters most

For a generated visual identity to have value, it should be **both recognizable and explainable**.

Suppose you build the creature renderer:

Code property	Possible creature feature
Number of major modules	Body segment count
Dependency density	Connection patterns or armor complexity
Structural depth	Vertical proportions
Detected dependency cycles	Knots, loops or tangled ornaments
Dominant languages	Material palette
Code churn	Animation intensity
Test coverage	Distinctive visual markings

These mappings are design ideas, not established scientific correspondences. They would need to be tested for readability.

The most important technical requirement is **stability**: small unrelated changes should not randomly transform the creature. Use normalized metrics, stable entity identifiers and deterministic seeds, while clearly defining what each feature represents.

If you get that right, the creature becomes more than decorative generative art. It becomes a recognizable fingerprint of a codebase that can evolve with the project.

8. My recommended starting sequence

Suggested research and MVP checklist 0/6

- [ ] **Run GitNexus on a real repository**
  Evaluate what relationships and clusters it extracts, and how easily you can consume the results.
- [ ] **Test CodeCharta**
  See how a structural city behaves, which metrics are understandable, and where it becomes visually overwhelming.

- [ ] **Try Eraser and CodeViz**
  Compare generated diagrams against actual repository structure and evaluate presentation quality.
- [ ] **Prototype Represent Graph**
  Use a canonical data format with source references and a clean interactive renderer.
- [ ] **Prototype one signature creature**
  Generate a deterministic, shareable visual identity for the same repository.
- [ ] **Compare two Git commits**
  Demonstrate how structural change translates into visible evolution.

Technical choices

For an MVP, I'd lean toward:

Layer	Starting choice
Parsing and graph extraction	Tree-sitter / language-specific analyzers; study GitNexus
Data format	Canonical graph JSON with stable IDs, metrics and provenance
2D graph exploration	Sigma.js + Graphology, or React Flow for smaller editable diagrams
Procedural 3D	Three.js + React Three Fiber
Layout	ELK.js for structured diagrams; custom layouts for cities and creatures
Visual generation	Deterministic grammar + seeded procedural geometry
AI integration	Claude Code / Codex skill for choosing representations and explaining them
Export	Self-contained HTML, SVG, PNG, animated video and eventually GLB

One caveat: inspect every project's license before incorporating its code. **Studying a project's ideas and UX is not the same as being permitted to reuse its implementation.**

Final recommendation

If I had to narrow all this research to just five GitHub repositories to inspect seriously:

01 Structural intelligence and code relationships

02 Geometry that communicates software metrics

03 Living, interactive world that changes over time

04	Agent-friendly, reusable visualization templates	
05	Deterministic procedural identity generation	

I would position Represent less as an alternative to GitDiagram and more as a **visual identity and exploration engine for structured systems**.

That's broader, much more distinctive, and it gives you a reason to create cities, creatures, blueprints, animated maps and 3D objects without forcing them all into the same interface.

I can also turn this research into a concrete **Represent MVP architecture and implementation roadmap**, identifying which open-source components to reuse and which parts are worth building yourself.